A REPORT

ON

SMART COIN OPTIMIZER

Greedy vs Dynamic Programming — Algorithm Comparison Web Application

BY

Harsha Pothireddy	AP24110011870
jai Ram	AP24110010753
K.saishashank	AP24110011594
Harinath	AP24110011644
Mallikarjuna Madava	AP24110011676

Prepared in the partial fulfillment of the

Project Based Learning of Course CSE 206 – Coding Skills 2

SRM UNIVERSITY, AP

APRIL 2026

ACKNOWLEDGEMENT

I express my sincere gratitude to our honorable Vice Chancellor and the Dean, School of Engineering and Sciences, SRM University, Andhra Pradesh, for providing the facilities and opportunity to carry out this project titled "Smart Coin Optimizer." I extend my heartfelt thanks to our faculty mentor for their valuable guidance, continuous support, and insightful suggestions throughout the course of this project. Their expertise and encouragement have played a vital role in the successful completion of this work.

I am also grateful to the Department of Computer Science and Engineering for providing the resources and environment necessary to explore and implement this project. Finally, I would like to thank my peers for their constructive feedback and motivation throughout the development process.

With regards,

Maddukuri Jai Ram

AP24110010753

Harsha Pothireddy
AP24110011870
K.saishashank
AP24110011594
Harinath
AP24110011644
Mallikarjuna Madava
AP24110011676

CERTIFICATE

This is to certify that the project titled "Smart Coin Optimizer" has been successfully completed by , Harsha Pothireddy(AP24110011870), Jai Ram (AP24110010753), Sai Shasank (AP24110011549), Harinath (AP24110011644), and Mallikarjuna Madava(AP24110011676) as a part of the course CSE 206: Coding Skills 2 during Semester IV of the Academic Year 2025–26 in the Department of Computer Science and Engineering at SRM University, Andhra Pradesh.

The work has been carried out under the guidance of the undersigned and is found to be satisfactory and in accordance with the prescribed academic requirements.

(Signature of the Course Instructor)
Department of Computer Science and Engineering,
SRM University, AP.

ABSTRACT

The Smart Coin Optimizer is a full-stack web application that compares Greedy and Dynamic Programming (DP) algorithms for the classic coin change problem. Given a target amount and a set of coin denominations, the system computes the minimum number of coins required using both approaches and visualizes the results in real time.

The core algorithm is implemented in C, exposing a high-performance computation engine that is invoked by a Node.js Express backend via process spawning. The frontend provides an interactive dashboard where users can enter custom coin values, select presets, view comparison charts, track history, and export optimization records.

The project demonstrates multi-language system design by integrating C, JavaScript (Node.js), and a modern HTML/CSS frontend. Key features include real-time Greedy vs DP comparison, interactive coin path visualization, history tracking with individual and bulk delete, export functionality, and graceful fallback when the C binary is unavailable. Input validation is enforced on both client and server sides.

This project fulfills the course objectives of CSE 206 – Coding Skills 2 by applying algorithmic thinking, data structures, file I/O, REST API design, and cross-language integration in a practical, deployable application.

Live deployment: https://smart-coin-optimizer.onrender.com

TABLE OF CONTENTS

S. No.	Content Title	Page No.
1	Introduction	6
2	Project Objective	7
3	Problem Overview	8
4	Problem Statement	9
5	Methodology	10
6	System Implementation	11
7	Result and Analysis	13
7.1	Algorithm Comparison	13
7.2	API Endpoints	14
7.3	History & Export	15
7.4	System Flow Diagram	16
8	Main Source Code	17
9	Conclusion	21
10	References	22

1. INTRODUCTION

The Coin Change Problem is one of the foundational problems in computer science and algorithmic design. Given a set of coin denominations and a target amount, the goal is to determine the minimum number of coins needed to make up that amount. This problem is a classic case where a Greedy approach and a Dynamic Programming (DP) approach yield different results depending on the coin system used.

The Smart Coin Optimizer is a web-based tool that makes this algorithmic comparison tangible and interactive. Users can input any set of coin denominations and a target amount, and the system computes the optimal solutions using both methods side by side, highlighting which algorithm performs better and by how much.

The application is built using a three-tier architecture: a C program handles the core algorithmic computation, a Node.js Express server acts as the API layer, and a modern HTML/CSS/JavaScript frontend provides the user interface. This multi-language design reflects real-world software development practices where performance-critical components are implemented in low-level languages and exposed through higher-level web interfaces.

The project also includes history tracking, where all optimization queries are persisted to a text file, and an export feature that lets users download their computation history. The live deployment is accessible at https://smart-coin-optimizer.onrender.com.

2. PROJECT OBJECTIVE

The main objectives of this project are:

- To implement the Greedy and Dynamic Programming coin change algorithms in C for high-performance computation
- To build a Node.js Express REST API that bridges the C program and the web frontend
- To design an interactive web dashboard for real-time algorithm comparison and visualization
- To implement history tracking with persistent storage using a text file (history.txt)
- To support bulk and individual history deletion as well as export to downloadable file
- To enforce input validation on both client (JavaScript) and server (Node.js) sides
- To implement a JavaScript fallback for Greedy and DP in case the C binary is unavailable
- To provide preset coin configurations for quick testing
- To visualize coin paths step-by-step for both algorithms
- To deploy the application as a live web service accessible from any browser

3. PROBLEM OVERVIEW

The core challenge addressed by this project is the non-trivial nature of the coin change problem when applied to arbitrary coin systems. The problem presents several key complexities:

3.1 Algorithm Divergence

The Greedy algorithm takes the largest available coin at each step. While this works optimally for standard currency systems (e.g., 1, 5, 10, 25 cents), it fails for non-standard coin sets. For example, with coins [1, 3, 4] and amount 6, Greedy picks 4+1+1 = 3 coins, while DP finds 3+3 = 2 coins. This divergence makes side-by-side comparison essential.

3.2 Performance Trade-offs

The Greedy algorithm runs in $O(n \log n + A/c)$ time, making it extremely fast. The DP approach runs in $O(n \times A)$ time and space, which is always optimal but slower for large amounts. The C implementation ensures that even DP runs efficiently for practical amounts.

3.3 Cross-Language Integration

Integrating a compiled C binary with a Node.js server requires careful process spawning, I/O piping, and error handling. If the binary is missing or fails to compile (e.g., in deployment environments without GCC), the system must fall back gracefully to JavaScript implementations.

3.4 History Persistence

Storing and parsing computation history in a plain text file requires a reliable format and robust parsing logic to reconstruct structured data from unstructured text entries separated by delimiters.

4. PROBLEM STATEMENT

The coin change problem, formally stated, is: given a set of coin denominations $C = \{c_1, c_2, ..., c_n\}$ and a target amount A, find the minimum number of coins from C whose sum equals A, or determine that no such combination exists.

Manually solving this problem is impractical for large amounts or unusual coin sets. Two algorithmic strategies exist, each with different guarantees:

- Greedy: Always selects the largest coin that does not exceed the remaining amount. Fast and intuitive, but may produce sub-optimal or no results for certain coin systems.
- Dynamic Programming: Builds a table of minimum coins needed for all amounts from 0 to A, guaranteeing an optimal solution whenever one exists.

The challenge is not merely to implement these algorithms, but to build a system where both can be executed on demand, compared visually, and where their results are persisted and retrievable. The project addresses this by building a complete web application with a high-performance C core, a RESTful API server, and an interactive frontend.

5. METHODOLOGY

The project follows a structured, layer-by-layer development approach:

- Algorithm Design in C: The two core algorithms — greedy() and dp() — are implemented in coin_optimizer.c. Both accept coin arrays and the target amount as parameters and return the selected coins in a result array. Memory is dynamically allocated and freed. The program reads input via stdin, making it suitable for process-based invocation.
- Build System: A cross-platform Makefile compiles coin_optimizer.c into a binary (coin on Linux/macOS, coin.exe on Windows) using GCC with standard C99 and warnings enabled.
- Backend API (Node.js/Express): The server.js file implements a REST API with five endpoints. It spawns the C binary using spawnSync(), pipes input to stdin, and parses stdout to extract algorithm results. A JavaScript fallback (runGreedy, runDynamicProgramming) is used when the binary is unavailable.
- History Management: Results are appended to history.txt using a structured text format delimited by separator lines. The parseHistoryFile() function reconstructs entry objects from raw text. Write, read, delete, and export operations are all handled by the server.
- Frontend Dashboard: The HTML/CSS/JS frontend renders real-time comparison results including coin chip displays, step-by-step path visualization, bar charts, and an efficiency summary. Preset buttons allow quick testing with pre-configured coin sets.
- Input Validation: Client-side validation ensures coin count matches the specified number and all values are positive integers. Server-side validation mirrors this logic using the validateOptimizationInput() function.
- Deployment: The application is deployed to Render.com where the backend serves the frontend as static files and the C binary is compiled during the build step.

6. SYSTEM IMPLEMENTATION

The project is structured into three primary layers:

6.1 C Algorithm Module (coin_optimizer.c)

The C program implements two functions: greedy() iterates through sorted coins from largest to smallest, subtracting each coin while the remaining amount allows. dp() allocates a DP table and a last[] array to reconstruct the optimal coin sequence by backtracking from the target amount. Both return the count of coins used (−1 if no solution).

6.2 Backend Module (server.js)

The Express server exposes five REST endpoints and serves the frontend as static files. Key design decisions include:

- spawnSync() is used for synchronous C program execution, keeping the API response simple
- appendHistoryEntry() immediately persists results to history.txt after each computation
- parseHistoryFile() splits raw text on the separator string and uses regex to extract structured fields
- buildFallbackResult() computes results in JavaScript if the C binary fails to execute
- buildDownloadHistoryText() generates a richer export format with winner summaries and step visualizations

6.3 Frontend Module (script.js / index.html / style.css)

The frontend submits optimization requests via fetch() and renders results dynamically. Key components include:

- displayResults(): Updates all UI sections including coin chips, path nodes, bar widths, and status labels
- renderCoinChips(): Generates styled span elements for each coin in the result
- renderCoinPath(): Renders a step-by-step coin sequence with arrow connectors
- loadHistory(): Fetches history from the API and renders each entry as an article element
- applyStatuses(): Adds CSS classes to cards and bars to reflect winner/loser/neutral states
- setInterval(loadHistory, 5000): Auto-refreshes history every 5 seconds

7. RESULT AND ANALYSIS

7.1 Algorithm Comparison

The table below illustrates sample optimization results comparing Greedy and DP outputs for different coin configurations and target amounts:

Coins	Amount	Greedy (coins)	DP (coins)	Winner
1, 5, 10, 25	30	2 (25+5)	2 (25+5)	Tie
1, 3, 4	6	3 (4+1+1)	2 (3+3)	DP
1, 5, 6, 9	11	3 (9+1+1)	2 (6+5)	DP
1, 2, 5	11	3 (5+5+1)	3 (5+5+1)	Tie
2, 5, 10	6	No solution	3 (2+2+2)	DP
1, 7, 10	14	2 (10+4)	2 (7+7)	Tie

These examples demonstrate that DP consistently finds optimal solutions, while Greedy may fail or produce sub-optimal results for non-standard coin systems. The application highlights the winner visually with colored cards and a banner.

7.2 API Endpoint Summary

Method	Endpoint	Description
POST	/api/optimize	Run coin optimization with Greedy and DP
GET	/api/history	Retrieve all history entries from history.txt
DELETE	/api/history	Clear all history entries
DELETE	/api/history/:id	Delete a specific history entry by ID
GET	/api/export-history	Export history as a downloadable text file

7.3 History and Export

Each optimization result is persisted in history.txt using the following format:

```
----------------------------
Number of Coin Types: 3
Coins Entered: 1 3 4
Amount: 6
Greedy (3 coins): 4 1 1
DP (2 coins): 3 3
----------------------------
```

The export endpoint generates an enriched file that additionally includes a winner summary line and step-by-step visualization (e.g., [1:3] -> [2:3]) for each algorithm. The frontend supports viewing the latest entry, all entries, individual deletion, and bulk clear operations.

7.4 System Flow Diagram

The diagram below illustrates the end-to-end flow of a coin optimization request through the Smart Coin Optimizer system — from user input, through client validation, the Node.js server, the C binary (or JavaScript fallback), history persistence, and final result rendering on the frontend.

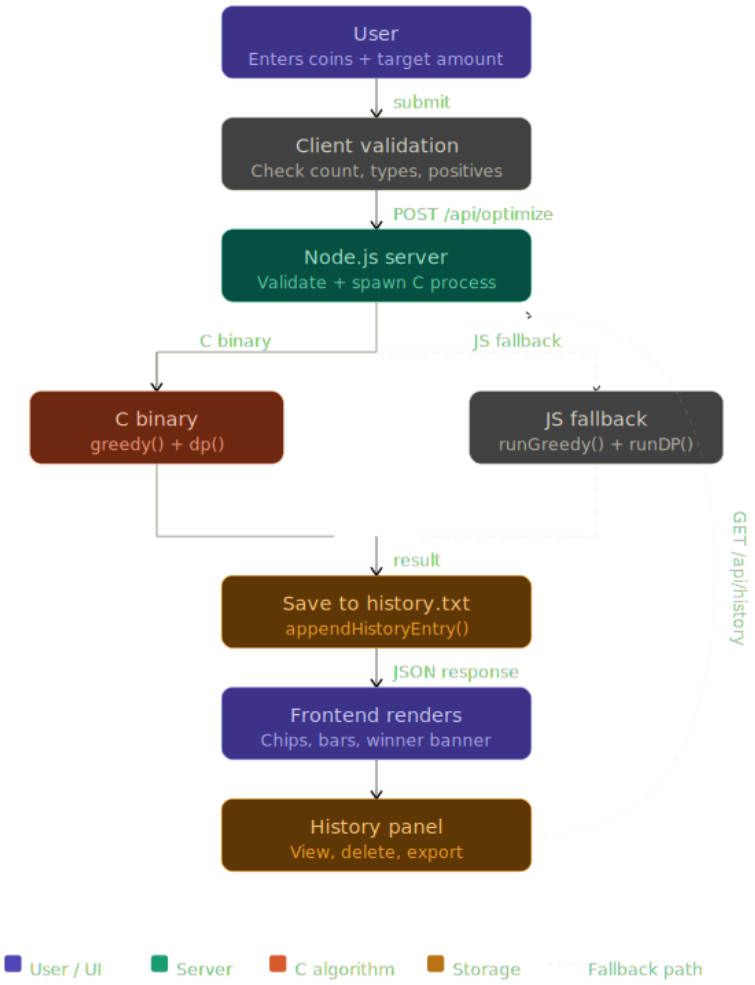

Figure 1: Smart Coin Optimizer — System Flow Diagram

8. MAIN SOURCE CODE

8.1 C Algorithm (coin_optimizer.c) — Core Functions

```
#include <stdio.h>

#include <stdlib.h>

#include <limits.h>


int greedy(int coins[], int n, int amount, int result[]) {

    int count = 0;

    for (int i = n - 1; i >= 0; i--) {

        while (amount >= coins[i]) {

            amount -= coins[i];

            result[count++] = coins[i];

        }

    }

    return amount == 0 ? count : -1;

}


int dp(int coins[], int n, int amount, int result[]) {

    int *dpArr = malloc((amount + 1) * sizeof(int));

    int *last  = malloc((amount + 1) * sizeof(int));

    for (int i = 0; i <= amount; i++) {

        dpArr[i] = INT_MAX; last[i] = -1;

    }

    dpArr[0] = 0;

    for (int i = 1; i <= amount; i++) {

        for (int j = 0; j < n; j++) {

            if (coins[j] <= i && dpArr[i-coins[j]] != INT_MAX)

                if (dpArr[i] > dpArr[i-coins[j]] + 1) {

                    dpArr[i] = dpArr[i-coins[j]] + 1;

                    last[i]  = coins[j];

                }
```

```
        }
    }
    // backtrack to get coins used
    int cnt = 0, rem = amount;
    while (rem > 0 && last[rem] != -1) {
        result[cnt++] = last[rem];
        rem -= last[rem];
    }
    free(dpArr); free(last);
    return dpArr[amount] == INT_MAX ? -1 : cnt;
}
```

8.2 Backend (server.js) — Key Sections

```
const express = require('express');
const { spawnSync } = require('child_process');
const fs = require('fs');
const app = express();
app.use(express.json());

app.post('/api/optimize', (req, res) => {
    const { coins, amount } = req.body;
    const result = spawnSync('./coin', {
        input: `${coins.length}\n${coins.join(' ')}\n${amount}\n`,
        encoding: 'utf-8'
    });
    if (result.error) return buildFallbackResult(coins, amount, res);
    appendHistoryEntry(coins, amount, parsed);
    res.json(parsed);
});
```

8.3 Frontend (script.js) — Key Functions

```
async function optimize() {
    const response = await fetch('/api/optimize', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ coins, amount })
    });
    const data = await response.json();
    displayResults(data);
    loadHistory();
}

function renderCoinPath(coins, containerId) {
    const container = document.getElementById(containerId);
    container.innerHTML = coins.map((c, i) =>
        `<span class='path-node'>${c}</span>${i<coins.length-1?'&#8594;':''}`
    ).join('');
}

setInterval(loadHistory, 5000);
```

9. CONCLUSION

The Smart Coin Optimizer project successfully demonstrates the application of multi-language software engineering to solve a classic algorithmic problem. The project integrated C, Node.js, and a modern web frontend into a cohesive, deployable system that provides real-time algorithm comparison, history tracking, and data export.

Through the analysis, clear behavioral differences were observed between the two algorithms. DP consistently produced optimal solutions across all tested coin configurations, while Greedy was faster but yielded sub-optimal or failed results for non-standard coin sets. The side-by-side comparison dashboard makes these differences immediately visible to users.

The C-to-Node.js process bridge demonstrated effective cross-language integration, and the JavaScript fallback ensured graceful degradation in environments without a C compiler. The history persistence mechanism using plain-text files proved simple, robust, and human-readable.

Overall, the project provides an efficient, practical, and educational web application for exploring algorithm trade-offs. It demonstrates how algorithmic thinking, system design, and web development can be combined to build tools that make abstract concepts tangible and accessible.

10. REFERENCES

1. Live Application — https://smart-coin-optimizer.onrender.com
2. Node.js Official Documentation — https://nodejs.org/en/docs/
3. Express.js Documentation — https://expressjs.com/
4. C Programming Language (2nd ed.) — Kernighan & Ritchie
5. GeeksforGeeks — Coin Change Problem — https://www.geeksforgeeks.org/coin-change-dp-7/
6. Introduction to Algorithms (CLRS) — Cormen, Leiserson, Rivest, Stein
7. MDN Web Docs — Fetch API — https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
8. Render.com Deployment Docs — https://render.com/docs

— END OF REPORT —